Optimizing a RAG Chatbot for JRU CSE Inquiries: Parameter Tests and CPU–GPU Performance

Luke Joaquin Bernardo
College of Computer Studies and Engineering
Jose Rizal University
Taguig City, Philippines
lukejoaquin.bernardo@my.jru.edu

John Cyruz Patron
College of Computer Studies and Engineering
Jose Rizal University
Mandaluyong City, Philippines
johncyruz.patron@my.jru.edu

Job Russel Enriquez
College of Computer Studies and Engineering
Jose Rizal University
Mandaluyong City, Philippines
jobrussel.enriquez@my.jru.edu

***Abstract*—We tested a retrieval-augmented generation chatbot for program and curriculum inquiries using the CSE sections of Jose Rizal University's College General Information for school year 2026–2027. We compared five chunking and retrieval configurations with Qwen2.5-1.5B-Instruct and a fixed set of 12 questions. We also measured three chunk sizes on actual CPU and T4 GPU runtimes, with a warm-up and three measured trials per configuration. Mean response times fell from 44.170–75.075 seconds on CPU to 3.381–4.890 seconds on GPU for the matched configurations. The response speedups were 13.063–15.352 times. A follow-up using an 800-character chunk size, 200-character overlap, top-k of three, and a revised prompt produced six fully correct, complete, and supported answers out of 12, compared with five for its original prompt. The ratings were checked against the fixed rubric and saved evidence. We recommend this GPU configuration for further development, but its remaining answer and retrieval errors prevent treating it as a reliable advising system.**

***Index Terms*—retrieval-augmented generation, curriculum inquiries, chunking, GPU benchmarking, answer quality**

I. Introduction

Prospective students may need to search several handbook sections to understand a program, course requirement, or prerequisite. Our group project focuses on this problem within JRU's College of Computer Studies and Engineering (CSE). For this activity, we used the 2026–2027 source edition and kept the scope within CSE. We do not present document statements as personalized or live enrollment advice.

Retrieval-augmented generation (RAG) combines document retrieval with language generation. The original RAG work describes using retrieved knowledge alongside a language model [1]. Our implementation follows the practical retrieve-then-generate pattern. We split the source into passages, embed them, retrieve relevant passages, and include them in the model's prompt. We did not fine-tune either model.

We followed the WW1–WW2 activity guide [2] to examine both answer quality and execution speed. Our questions were: Which tested chunking and retrieval configuration offers a useful balance for this source? How much faster are matched workloads in the GPU runtime? Can a revised prompt or prepared source improve the answers? This activity is a small optimization study, rather than the full evaluation planned in our original proposal.

II. Experimental Setup

A. Source Documents and Test Questions

Our knowledge base came from the university's College General Information, school year 2026–2027 [3]. The supplied CSE-only PDF has 64 pages, including an image-only cover retained for provenance. The original-text pipeline used the remaining 63 readable pages. The package includes a map between filtered, original PDF, and printed page numbers. Source inconsistencies were preserved rather than corrected by assumption.

We used 12 fixed, source-answerable questions. The set covers Computer Engineering, Electronics Engineering, the two Entertainment and Multimedia Computing specializations, Information Technology, and Information Technology with Business Analytics. It includes four easy, four average, and four difficult questions. The questions ask about program focus, curriculum placement, prerequisites, units, and comparisons across sections or courses.

We kept the same question wording, reference answers, and scoring rubric across the compared runs. The key contains 32 documentary evidence units and two additional answer qualifications. The evaluation files were kept outside the vector store and generation prompts. This prevents the chatbot from retrieving its own answer key. All questions are answerable, so this experiment does not measure refusal behavior for unsupported inquiries.

B. Models, Retrieval, and Generation

We used `Qwen/Qwen2.5-1.5B-Instruct` [4] with `sentence-transformers/all-MiniLM-L6-v2` [5]. The language model revision begins `989aa7980e4c`, and the embedding revision begins `1110a243fdf4`. Their complete revision identifiers are stored in the submitted data. We used Chroma for vector storage and the template's recursive character splitter. Chunk size and overlap are character counts, not token counts.

For the controlled CPU/GPU comparison, we kept float32, eager attention, greedy decoding, a 256-new-token limit, seed 42, and two Torch CPU threads. We checked the actual devices of both the embedding and language models. They used `cpu`

TABLE I
TESTED ORIGINAL-PROMPT CONFIGURATIONS

Run	Chunk	Overlap	Top-k	Devices
A	400	50	3	CPU, GPU
B	800	100	3	CPU, GPU
C	1200	150	3	CPU, GPU
D	800	200	3	GPU
E	800	100	5	GPU

in the CPU runtime and `cuda:0` in the GPU runtime. No external model API was used for these measurements.

The recorded software versions were Torch 2.11.0, Transformers 5.22.1, Sentence Transformers 6.2.0, LangChain Core 0.3.86, LangChain Chroma 0.2.6, ChromaDB 1.5.9, and pypdf 6.10.0. Torch had CPU and CUDA build suffixes in the corresponding runtimes. We retained those full versions in the records while normalizing only the build suffix for the comparison check.

C. Hardware and Configuration Matrix

The CPU runtime reported an AMD EPYC 7B12 host. The GPU runtime reported a Tesla T4 and an Intel Xeon host at 2.00 GHz. These are separate Colab runtime environments. We restarted when switching hardware and verified model placement. The host difference remains a limitation when interpreting the speedups.

We tested A–E on GPU for parameter selection. We tested A, B, and C on CPU as the required small, medium, and large chunk workloads. B versus D changes overlap, while B versus E changes top-k. A, B, and C change both chunk size and overlap, so their differences cannot be attributed to chunk size alone.

D. Repeated Trials and Timing Boundaries

Each configuration/device pair had one unscored warm-up using the first question, followed by three measured trials over all 12 questions. We rebuilt the index for each trial. Each pair therefore contributes three indexing observations and 36 measured response observations. The original-prompt experiment has 288 measured responses. Two GPU follow-ups add 72, giving 360 current measured responses overall.

Indexing time includes chunk creation, embedding, and vector-index construction. The raw trial log also separates chunking from embedding and index creation. Retrieval time includes retrieving the passages for a question. Generation time measures the language model call, with GPU synchronization around the timed operation. Total response time includes retrieval, prompt preparation, generation, and decoding:

$$t_{\text{response}} = t_{\text{retrieval}} + t_{\text{preparation}} + t_{\text{generation}} + t_{\text{decoding}}. \quad (1)$$

Model loading, source loading, grading, and export are outside response timing. We averaged indexing across the three measured builds and response stages across the 36 question observations. Warm-ups, diagnostics, and incomplete attempts were excluded. We retained setup costs separately because a short interactive session still has to pay them.

III. Answer and Retrieval Evaluation

We used correctness, source support, and completeness as separate criteria. Correctness is scored from zero to two against the source and reference answer. A score of two means that all substantive factual claims made are correct; it does not guarantee that the answer is complete. Source support is also scored from zero to two, but against the passages actually supplied to the model. A fact can match the handbook and still be unsupported by the retrieved evidence.

Completeness is the correctly answered required-element count divided by the fixed count for that question. A complete prerequisite or requested course-code set counts as one grouped element. Missing a member leaves that element incomplete. We call an answer fully successful only when correctness and support are both two and completeness is one. We report successful questions out of 12, rather than treating repeated identical answers as independent successes.

For retrieval, precision at k is the fraction of returned passages that help answer at least one requested part. Evidence coverage is the fraction of required documentary facts fully supported by the union of those passages. A shared keyword alone is insufficient. Repeated passages do not increase coverage. The notebook checks that the configured number of passages was returned and records the actual full prompt and source locations.

Our group verified the initial ratings. A subsequent source audit checked all 84 unique answer/context groups against the fixed rubric, reference answers, and actual retrieved evidence. It corrected one source-support score where an answer named the wrong source passage. A judgment is reused only when the question, answer, full prompt, and ordered retrieved passages match exactly. Every repetition retains its own timing row. Automatic source and record checks establish file consistency and arithmetic, but they do not establish semantic correctness. Repeated identical answers share a rating and are not counted as independent reviews.

IV. Results and Discussion

We took the measured values in this report from the executed notebook outputs. We included the raw trial records and exported tables in the experimental data ZIP. The Excel workbook calculates averages and speedups from those records. Table II reports the current results, rounded for readability.

A. CPU Versus GPU Performance

We calculated each speedup using the CPU and GPU means from the same original-prompt configuration:

$$S = \frac{\bar{t}_{\text{CPU}}}{\bar{t}_{\text{GPU}}}. \quad (2)$$

For A, the unrounded response means give $44.169617/3.381341 = 13.063$. The result means that the GPU runtime processed this response workload about 13

TABLE II
CURRENT MEAN TIMES FROM THE EXECUTED NOTEBOOK, IN SECONDS

Experiment	Device/run	Indexing	Retrieval	Generation	Total response	Output tokens	Measured answers
Original prompt	CPU A	12.805	0.0460	44.121	44.170	72.25	36
Original prompt	CPU B	14.026	0.0430	57.803	57.849	76.50	36
Original prompt	CPU C	13.288	0.0319	75.039	75.075	89.58	36
Original prompt	GPU A	1.002	0.0135	3.365	3.381	72.25	36
Original prompt	GPU B	0.855	0.0131	4.042	4.058	76.50	36
Original prompt	GPU C	0.819	0.0128	4.873	4.890	89.58	36
Original prompt	GPU D	0.995	0.0136	4.301	4.319	83.67	36
Original prompt	GPU E	0.861	0.0135	5.572	5.590	101.58	36
Revised prompt	GPU D	1.045	0.0138	3.725	3.743	71.25	36
Prepared source	GPU D	2.145	0.0127	4.163	4.179	88.42	36

TABLE III
MATCHED CPU/GPU SPEEDUPS FOR THE ORIGINAL PROMPT

Run	Indexing	Retrieval	Generation	Response
A	12.780	3.413	13.111	13.063
B	16.396	3.292	14.302	14.254
C	16.222	2.495	15.400	15.352

TABLE IV
SOURCE-AUDITED QUALITY OF ORIGINAL-PROMPT GPU ANSWERS

Run	Corr. /2	Supp. /2	Compl. %	P@k %	Coverage %	Full /12
A	1.08	0.75	47.9	25.0	35.4	3
B	1.33	1.17	62.5	50.0	61.8	3
C	1.42	1.33	60.4	44.4	68.1	4
D	1.33	1.25	64.6	50.0	63.2	5
E	1.33	1.42	54.2	33.3	70.1	4

times faster. It is not a speedup for model downloading or the entire notebook.

Generation dominated total response time on both devices. For CPU C, mean generation was 75.039 seconds out of a 75.075-second response. For GPU C, it was 4.873 out of 4.890 seconds. The large generation reduction explains most of the response improvement. Retrieval remained a small part of the total, so its smaller speedup had little effect on the overall result.

Indexing also became faster, but it is a different workload from answering a question. Original B indexing averaged 14.026 seconds on CPU and 0.855 seconds on GPU. We do not add an entire index rebuild to each response because the operational chatbot can reuse an existing index. Rebuilding in the experiment gives us comparable indexing measurements.

Setup still matters. Language-model loading was 77.680 seconds in the original CPU session and 37.555 seconds in the original GPU session. Embedding-model loading was 15.668 and 18.921 seconds, respectively. These are recorded session costs, not three-trial setup averages. We did not find that every setup stage improved on GPU, and we cannot infer cold-start performance from response time alone.

B. Original Configuration Tradeoffs

A was fastest on GPU at 3.381 seconds, but only three questions met all success criteria. C reached four and D reached five. D used less response time than C, at 4.319 versus 4.890 seconds, and had higher completeness. C had stronger mean correctness, support, and documentary coverage. These tradeoffs made D a useful follow-up candidate rather than an unqualified winner on every metric.

Increasing overlap from 100 to 200 characters in B versus D kept chunk size and top-k fixed. Completeness rose from 62.5% to 64.6%, and fully successful questions rose from three to five. Mean response time increased from 4.058 to 4.319 seconds. For this source, the additional overlap offered a modest quality gain at a small time cost.

Increasing top-k from three to five in B versus E gave a different tradeoff. Documentary coverage rose from 61.8% to 70.1%, but precision fell from 50.0% to 33.3%. Completeness fell from 62.5% to 54.2%, while response time rose to 5.590 seconds. More retrieved material did not consistently help the model use the correct course information. E also generated more output tokens on average, so the extra time is not solely a retrieval cost.

C. Revised Prompt and Prepared Source

The revised prompt adds explicit instructions to match facts to the named course and program, avoid neighboring course details, and answer comparison items separately. It also asks the model to identify a specific missing detail rather than fill it in. We kept D's retrieval configuration and the original PDF extraction for this follow-up.

The prepared-source follow-up used that same revised prompt and D settings. We changed the source representation to 1,094 labelled records: 341 curriculum rows, 451 group-requirement rows, 180 course descriptions, 70 narratives, 51 semester summaries, and one heading. The preparation retained page references and source conflicts. It joined whole course entries and changed content ordering before applying the same recursive splitter. This is a combined preparation strategy, not an isolated test of one formatting change.

The revised prompt improved one additional full answer. It also reduced mean output length from 83.67 to 71.25 tokens.

TABLE V
CURRENT GPU D FOLLOW-UPS AND SOURCE-AUDITED QUALITY

Variant	Response s	Corr. /2	Supp. /2	Compl. %	Full /12
Original prompt	4.319	1.33	1.25	64.6	5
Revised prompt	3.743	1.50	1.33	70.8	6
Prepared source	4.179	1.33	1.25	60.4	5

Its shorter measured response time is consistent with shorter generation, but separate Colab runs do not establish that the prompt alone caused the time reduction. Retrieval precision and evidence coverage stayed at 50.0% and 63.2%, so this prompt did not fix missing retrieved evidence.

The prepared source produced both gains and regressions. For example, it improved coverage of parts of the Game Programming comparison, but it did not correctly answer the ECE C405 lecture-unit question. Its precision fell to 36.1%, and mean completeness fell to 60.4%. We therefore do not recommend it over the revised-prompt PDF pipeline based on these results. Organizing source records differently can change which passages rank highly; cleaner records do not automatically produce better retrieval.

D. Answer-Level Observations

The lecture-unit question shows why we separated correctness from support. Some answers stated the correct ECE C405 lecture value, but their curriculum passage only established a unit value without the lecture label. Those answers received full correctness and no source support. Configuration C had the explicit course description in Source 3, but its answer cited Source 2, which described other courses. The final audit lowered its support score from two to one. This reduced C from five to four fully successful questions without changing its fact correctness, completeness, or timing.

The Computer Architecture question exposed a source conflict. The year-by-year curriculum and course description list four units, while the group-requirements listing gives two. A complete answer must preserve that 4/2/4 distinction and identify the unresolved disagreement. Neither substituting another course's prerequisite nor guessing which listing is correct resolves this question.

The revised prompt gave a fully successful answer to the ITC C402 Internship question where the original D answer was incomplete. But several comparisons still lacked complete prerequisites, units, or the correct section associations. These examples support a limited development recommendation. They do not support claiming that the chatbot is ready to answer all CSE inquiries accurately.

E. Earlier Results and the Qwen3 Attempt

Our latest revised-prompt D run averaged 3.743 seconds. An earlier completed run averaged 3.838 seconds and produced exactly matching question, answer, full-prompt, and retrieved-context records. Its prepared quality judgments could therefore be reused. We preserved those earlier trials as history and excluded them from the current means. An earlier rerun is not a separate experimental variant.

We also retained a supplementary deployment test of the proposed Qwen3-8B model using four-bit NF4 and nonthinking mode. Its GPU A–E benchmarks completed, with mean response times of 8.570, 11.320, 12.071, 9.864, and 14.473 seconds, respectively. The CPU pilots were interrupted before completing an answer. They provide no completed CPU response average and no valid Qwen3 CPU/GPU speedup.

We treat that attempt as a feasibility finding. Model size, quantization, and some software settings differ from the main experiment, so it is not a controlled model-only comparison. The records do not establish a quality advantage for Qwen3. We moved to the smaller Qwen2.5 model to complete matched CPU/GPU workloads, and kept the unsuccessful CPU attempts visible rather than assigning them invented response times.

V. RECOMMENDED CONFIGURATION

We recommend GPU D with the original PDF extraction, 800-character chunks, 200-character overlap, top-k of three, and the revised prompt as our next development baseline. It completed the measured runs on the available T4 and gave the best observed balance among the tested follow-ups: a 3.743-second mean response, 70.8% completeness, and six fully successful questions.

We prefer this baseline to A because A's shorter response time came with weaker answer quality. We prefer it to E because retrieving five passages added time without a consistent answer benefit. We also prefer it to the tested prepared-source strategy, which lost some retrieval precision and completeness. For a live service, the remaining six unsuccessful questions would require better evidence retrieval and careful handling of missing or conflicting information.

The GPU benefit is practical for repeated questions because generation dominates this workload. Model loading and GPU availability still matter for short sessions. We observed completed runs, but did not conduct a peak-memory or concurrency study. We therefore make no measured claim about maximum user capacity, memory savings, or deployment cost.

VI. LIMITATIONS AND POSSIBLE IMPROVEMENTS

Our benchmark has only 12 answerable questions from one handbook edition. Repeated trials improve timing observations, but they do not create new independent questions. We also inspected this set while developing the follow-ups. Their quality gains are development-set findings, not evidence of generalization to unseen inquiries. The initial ratings were group verified and then source-audited, but we have not measured inter-rater agreement.

Colab sessions can differ in host hardware, load, caches, and setup conditions. Our measured CPU and GPU hosts differed. We used matched sources, questions, models, settings, and timing boundaries, but this remains a comparison of observed Colab environments. We report means without claiming

statistical significance. The different output lengths across parameter configurations also affect response time.

The source includes flattened tables, page continuations, similar course codes, and unresolved document conflicts. These can cause retrieval or course-association errors. The embedding model's default input limit is 256 word pieces [5], so a larger character chunk may not have its entire text represented in the embedding. Character chunk size is therefore not a direct measure of the information effectively embedded.

Our next step is to evaluate on a separate held-out question set. We would test hybrid retrieval or reranking, stronger program and course metadata, and small controlled changes to source preparation. Each change should preserve the same evaluation rules. We would also add unsupported inquiries and check that the model names a missing fact or unresolved conflict instead of guessing.

VII. Reproducibility and Submission Files

Our submitted notebook keeps the template's loading, indexing, model, question-answering, and testing flow. Section 2 selects the experiment, device, and configurations. We upload the submitted experimental data ZIP in Section 3. The main, revised-prompt, and prepared-source settings can be run separately using their recorded protocols. Section 8 performs the diagnostic and repeated trials, and Section 10 recalculates the consolidated tables from saved records. Its final tables and evidence panels were recalculated after the source audit; the model-generated answers and timings were retained. The latest execution freshly measured revised-prompt GPU D; the other displayed experiments were restored from their actual earlier notebook records, not newly timed in that execution.

The data archive contains the source PDF, fixed questions and key, source-page references, prepared records and preparation code, raw trials, preserved earlier runs, evaluation judgments, pinned revisions, and supplementary records. Working indexes and model weights are rebuilt. The Excel workbook retains raw response and indexing rows, calculated summaries, speedups, answers, scoring reasons, and supporting passages. Warm-ups, historical records, and supplementary attempts are labelled separately. The submitted executed notebook retains its visible outputs.

Exact measured times need not repeat on a later Colab session. Pinned revisions, fixed decoding settings, source checks, and recorded prompts make the procedure traceable, but they do not guarantee identical timing or bitwise-identical outputs across every backend. A fresh answer with different content or context requires a new quality judgment rather than automatic reuse.

VIII. Conclusion

We found a clear GPU response-time advantage for the three matched chunk workloads. Their observed speedups ranged from 13.063 to 15.352 times, mainly because generation became faster. Parameter selection still required checking answer quality: the fastest configuration was not the most useful one for our questions.

Our recommendation is revised-prompt GPU D. It improved the observed balance of completeness and response time, while source preparation did not consistently improve the answers. The remaining errors show that hardware speed and grounded answer quality are separate problems. We will need unseen-question testing before making a broader reliability claim.

References

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[2] Jose Rizal University, "ITC-C508 Lab Exercise WW1 and WW2 (Optimization and GPU Testing): Student Experiment Guide," course handout, n.d.

[3] Jose Rizal University, *College General Information, School Year 2026–2027*. Institutional handbook, CSE sections. Submitted source: `COL-GI-2026-2027_CSE_Only.pdf`, 64-page filtered excerpt with original-page map.

[4] Qwen Team, "Qwen2.5-1.5B-Instruct," model card, 2024. Accessed: Nov. 4, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

[5] Sentence Transformers, "all-MiniLM-L6-v2," model card. Accessed: Nov. 4, 2026. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2